

Projet : Analyse Statique d'une Application Java avec une Interface Graphique

Lien dépôt : <https://github.com/nguyen703/Analyse-logiciel-TP1>

Lien démo :

TP1: <https://drive.google.com/file/d/1wmeup0skjJY02dHnDa-IZ5M8q0fAtg8k/view?usp=sharing>

TP2: <https://drive.google.com/file/d/1QG2syALo9HIHgWGUSbo01g-l5YiXYgae/view?usp=sharing>

1. Structure générale du projet

L'architecture du projet suit une organisation claire et modulaire :

`com.hnguyen703.AnalyzerUI`

Point d'entrée de l'application, gère l'interface graphique avec **JavaFX**. JavaFX est utilisé pour créer une interface utilisateur interactive permettant de visualiser les résultats de l'analyse statique.

`com.hnguyen703.AnalyzerSpoonUI`

[TP2] Point d'entrée alternatif utilisant **Spoon** comme framework d'analyse au lieu de JDT. Fournit les mêmes fonctionnalités d'interface graphique avec une implémentation basée sur Spoon pour l'analyse du code.

`com.hnguyen703.analyzer`

Contient la logique principale d'analyse du code :

- **Analyzer** : classe principale qui parcourt les fichiers sources Java et agrège les informations collectées.
- Utilise les classes *visitors* pour parcourir l'arbre syntaxique (AST).
- **[TP2]** Calcule les métriques de couplage entre classes et construit un dendrogramme pour l'analyse de clustering hiérarchique.

`com.hnguyen703.analyzer.SpoonAnalyzer`

[TP2] Implémentation alternative de l'analyseur utilisant le framework **Spoon**. Fournit les mêmes capacités d'analyse que **Analyzer** mais avec l'API Spoon pour la manipulation de l'AST. Inclut également les fonctionnalités de couplage et de clustering.

com.hnguyen703.analyzer.visitors

Regroupe les visiteurs JDT responsables de l'extraction des informations :

- **TypeDeclarationVisitor** : détecte les classes et interfaces.
- **MethodDeclarationVisitor** : recense les méthodes, leurs signatures et leurs lignes de code.
- **FieldAccessVisitor** et **VariableDeclarationFragmentVisitor** : identifient les attributs.
- **MethodInvocationVisitor** : repère les appels de méthodes (pour le graphe d'appel).

com.hnguyen703.ui

Regroupe tous les éléments liés à l'interface graphique.

ui.components

- **StatisticsView** : affiche les statistiques calculées (nombre de classes, méthodes, moyennes, etc.).
- **CallGraphView** : visualise le graphe d'appel à l'aide de **GraphStream**.
- **CouplingGraphView** : **[TP2]** affiche le graphe de couplage pondéré entre les classes avec un panneau textuel détaillant les relations de couplage et leurs poids.
- **SummaryPanel** : propose une vue synthétique du projet analysé.

ui.models

- **ClassStat**, **MethodStat** : structures de données stockant les résultats (nom, nombre de méthodes, lignes, attributs...).

ui.utils

- **FileUtils** : gère la lecture des fichiers et répertoires.
- **Constants** : centralise les chemins et paramètres.

dossier.test

Regroupe les classes de test :

- `MainTest`, `SampleTest`, `TestRunner` pour vérifier la fiabilité de l'analyse et de l'affichage.
-

2. Fonctionnement de l'analyse

1. **Lecture du projet cible** : `Analyzer` parcourt récursivement les fichiers `.java`.
2. **Visite de l'AST** : les visiteurs extraient les informations sur les classes, attributs, méthodes et appels.
3. **Calcul des statistiques** :
 - Nombre de classes, méthodes, packages.
 - Moyennes (méthodes/classe, lignes/méthode, attributs/classe).
 - Classes ayant le plus de méthodes ou d'attributs.
4. **Construction du graphe d'appel** :
 - Les relations méthode → méthode sont enregistrées par `MethodInvocationVisitor`.
 - `CallGraphView` affiche ces liens sous forme de graphe orienté.
5. **[TP2] Analyse de couplage** :
 - Calcul d'une métrique de couplage pondérée entre paires de classes : $\text{Couplage}(A, B) = \frac{\text{Relations}(A, B)}{\text{TotalRelations}}$.
 - Construction d'une matrice de couplage pour toutes les classes du projet.
 - Visualisation du graphe de couplage avec coloration selon l'intensité (forte, moyenne, faible).
6. **[TP2] Clustering hiérarchique** :
 - Construction d'un dendrogramme par regroupement itératif basé sur le couplage maximum.
 - Identification automatique de modules cohésifs selon un seuil de couplage interne (CP).
 - Respect de la contrainte : maximum $M/2$ modules pour M classes.

3. Interface graphique

L'interface est construite autour de `AnalyzerUI` (JDT) et `AnalyzerSpoonUI` (Spoon), composée de trois onglets principaux :

- **Onglet "Statistics"** → affiche les résultats numériques et les classes les plus importantes.
- **Onglet "Call Graph"** → montre visuellement les appels entre méthodes via `GraphStream`.
- **[TP2] Onglet "Coupling Graph"** → affiche le graphe de couplage pondéré entre classes avec :
 - Visualisation graphique interactive (GraphStream).
 - Panneau textuel détaillant toutes les relations de couplage triées par poids.
 - Statistiques de couplage (total, moyenne, relation la plus forte).
 - Code couleur :  Rouge (>10%),  Orange (5-10%),  Vert (<5%).

L'utilisateur peut charger un dossier Java, lancer l'analyse et visualiser instantanément :

- la complexité du projet,
 - les zones fortement couplées,
 - les classes principales de l'architecture,
 - **[TP2]** les métriques de couplage entre classes,
 - **[TP2]** les modules identifiés par clustering hiérarchique.
-

4. Implémentations alternatives (TP2)

Le projet offre deux implémentations complètes de l'analyse :

Avec JDT (Eclipse AST)

- Classes : `Analyzer` , `AnalyzerUI`
- Utilise les visiteurs JDT pour parcourir l'AST
- Point d'entrée : `AnalyzerUI.main()`

Avec Spoon

- Classes : `SpoonAnalyzer` , `AnalyzerSpoonUI`
- Utilise l'API Spoon pour la manipulation de l'AST
- Point d'entrée : `AnalyzerSpoonUI.main()`

Les deux implémentations fournissent des résultats identiques et partagent la même interface utilisateur (JavaFX + GraphStream).